

Chapter 1 - Introduction to Algorithms (Recap)

Goal of This Chapter

The first chapter introduces the idea that:

The choice of algorithm often matters more than the speed of the computer.

A bad algorithm on a fast computer can be slower than a good algorithm on a slow computer.

What is an Algorithm?

An algorithm is a sequence of steps used to solve a problem.

Examples:

- Find a name in a phone book
- Sort a list of numbers
- Search for a file
- Find the shortest route on a map

The main question is:

How does the running time grow when the amount of data grows?

Binary Search

Imagine a phone book with 1,000 names.

You need to find:

Sergio

There are two approaches.

Linear Search

Check every name one by one.

Alice
Bob

```
Charlie
David
...
Sergio
```

Worst case:

1000 checks

Complexity:

$O(n)$

where:

n = number of elements

Binary Search

Requirements:

- Data must be sorted.
- At every step, discard half of the remaining data.

Example:

1 2 3 4 5 6 7 8 9

Search for:

8

Steps:

```
Middle = 5
8 > 5
Discard left half

Middle = 7
8 > 7
Discard left half
```

Middle = 8
Found

Only 3 comparisons.

Understanding Logarithms

Most beginners struggle here.

Forget formulas.

Think:

How many times can I divide something by 2 before reaching 1?

Example:

16

16 -> 8 -> 4 -> 2 -> 1

4 divisions.

Therefore:

$$\log_2(16) = 4$$

Example:

32

32 -> 16 -> 8 -> 4 -> 2 -> 1

5 divisions.

Therefore:

$$\log_2(32) = 5$$

Why $O(\log n)$ is Crazy Fast

Let's compare.

Elements	$O(n)$	$O(\log n)$
10	10	4
100	100	7
1,000	1,000	10
1,000,000	1,000,000	20
1,000,000,000	1,000,000,000	30

Notice something shocking:

Increasing data from:

1 million

to

1 billion

adds only:

10 extra operations

for binary search.

Big O Notation

Big O measures how running time grows.

It does not measure exact seconds.

Instead, it measures growth.

$O(1)$

Constant time.

Example:

```
value := arr[5]
```

No matter if the array contains:

```
10 elements  
or  
10 million elements
```

The operation count remains the same.

$O(\log n)$

Examples:

- Binary Search
- Balanced Trees
- Database Indexes
- DNS Lookups

At every step:

```
cut the search space in half
```

$O(n)$

Linear time.

Example:

```
for _, item := range arr {  
    fmt.Println(item)  
}
```

Every element must be visited.

Visual Comparison

$O(1)$

```
n = 10          -> 1 operation
n = 1,000       -> 1 operation
n = 1,000,000   -> 1 operation
```

$O(\log n)$

```
n = 10          -> ~4 operations
n = 1,000       -> ~10 operations
n = 1,000,000   -> ~20 operations
```

$O(n)$

```
n = 10          -> 10 operations
n = 1,000       -> 1,000 operations
n = 1,000,000   -> 1,000,000 operations
```

Binary Search in Go

```
func binarySearch(arr []int, target int) int {
    low := 0
    high := len(arr) - 1

    for low <= high {
        mid := low + (high-low)/2
        guess := arr[mid]

        if guess == target {
            return mid
        }

        if guess > target {
            high = mid - 1
        } else {
            low = mid + 1
        }
    }
}
```

```
    return -1  
}
```

Complexity:

$O(\log n)$

Mental Model to Remember

$O(1)$

"I know exactly where it is."

$O(\log n)$

"Every step removes half of the problem."

$O(n)$

"I must inspect everything."

Key Takeaways

1. Algorithms matter more than hardware.
2. Binary Search requires sorted data.
3. Binary Search runs in $O(\log n)$.
4. Linear Search runs in $O(n)$.
5. $O(1)$ is constant time.
6. Big O describes growth, not exact time.
7. The larger the dataset becomes, the more important algorithm choice becomes.